

# Restful API — 复习要点与考点总结

## 前后端分离的基本概念 (Frontend-Backend Separation)

### 第一阶段: JSP/PHP 时代 (JSP/PHP Era)

[p.5]

- **优点 (Advantages):** 开发简单 (一个 JSP 文件搞定); 部署方便 (Easy Deployment); SEO 友好 (服务端渲染, Server-Side Rendering); 首屏速度快 (Fast First Screen)。
- **缺点 (Disadvantages):** 职责混乱 (Java + HTML + CSS + JS 混在一起); 难以维护 (牵一发而动全身); 前端无法独立开发 (依赖后端环境); 无法复用 (每个页面都要重新渲染)。 [p.5]

### 第二阶段: MVC 框架时代 (MVC Framework Era)

[p.6]

- **优点 (Advantages):** 职责分离 (Controller – Service – Dao); 可测试性提升 (可以单元测试, Unit Test); 代码复用 (Service 层可被多个 Controller 调用)。 [p.6]
- **缺点 (Disadvantages):** 页面跳转由后端控制; 静态资源混合部署 (性能瓶颈, Performance Bottleneck); 无法适应移动端 (一套模板打天下); 前端仍然依赖后端。 [p.6]

### 第三阶段: 前后端分离时代 (Decoupled Frontend-Backend Era)

[p.7–11]

#### 为什么需要前后端分离? (Why Frontend-Backend Separation?)

[p.7]

- **传统单体架构的局限 (Monolithic Architecture Limitations):** 传统 Web 应用中, 渲染页面、业务逻辑和数据访问都在同一代码库中运行。这种结构起步快, 但随着系统复杂度增加, 会出现扩展困难、上线周期长、前端无法独立迭代等问题。 [p.7]
- **不同客户端需求的冲突 (Multi-Client Conflict):** 如果既有桌面端 (Desktop) 又有移动端 (Mobile), 单一后端需要频繁兼顾不同终端的接口需求, 造成开发瓶颈和协调成本。 [p.7]

#### 什么是前后端分离? (What is Frontend-Backend Separation?)

[p.8]

- **核心理念 (Core Concept):** 把用户界面层 (前端, UI Layer) 和业务数据层 (后端, Business/Data Layer) 解耦 (Decouple), 双方通过统一的 API 通信。 [p.8]
- 前端代码与后端代码在不同的服务器实例上运行, 后端只暴露数据接口, 不再负责渲染页面。前端通常通过 RESTful API 调用后端获取数据。 [p.8]
- 这种架构也被称为 “headless” (无头) 或 “decoupled” (解耦) 架构。 [p.8]

#### 前后端分离的优势 (Advantages)

[p.9]

- **灵活的技术栈和独立部署 (Flexible Tech Stack & Independent Deployment):** 前端团队可以选择适合业务的框架 (Vue、React 等), 后端可以使用适合的语言和框架, 两个服务独立部署, 互不影响。 [p.9]

- **缩短迭代周期、提升协作效率 (Shorter Iteration & Better Collaboration):** 前端开发者无需深入后端逻辑就能开发功能，可以通过模拟接口 (Mock API) 进行并行开发。后端变更只需保证 API 契约 (API Contract) 不变，前端不会受到影响。 [p.9]
- **面向多端的可扩展性 (Multi-Client Extensibility):** 通过为不同终端提供专属的 Backend-for-Frontend (BFF) 层，可以根据桌面端、移动端等界面的不同需求定制接口，实现性能优化和体验差异化。 [p.9]
- **更容易实施微服务和云原生 (Microservices & Cloud Native Ready):** 微服务强调服务粒度，前后端分离为微服务拆分提供了天然接口界限，也方便通过 API 网关 (API Gateway) 统一治理。 [p.9]

## Web 服务的实现方式 (Web Service Implementation)

### Web Service 简介 (Introduction to Web Service)

[p.13]

- Web Service (Web 服务) 是一种基于 Web 的远程调用 (Remote Call) 技术，主要用于不同系统之间的互操作 (Interoperability)。 [p.13]
- **核心理念:**实现“基于 Web 无缝集成”的概念;实现不同系统间的相互调用(语言无关, Language-independent、平台无关, Platform-independent) ——Anywhere、Anytime、AnyDevice。 [p.13]
- Web Service 有多种不同的实现方式:**SOAP** (Simple Object Access Protocol, 简单对象访问协议) 和 **REST** (Representational State Transfer, 表述性状态转移)。 [p.13]

### 基于 SOAP 的 Web Service (SOAP-based Web Service)

[p.15–18]

- **基于 SOAP 和 XML:** 传统 Web Service 主要使用 SOAP 协议，并依赖 XML 进行数据传输。 [p.15]
- **WSDL (Web Service Description Language):** Web 服务描述语言，提供服务描述，定义可用的方法和数据格式，通常用于生成客户端代码。 [p.15]
- **UDDI (Universal Description, Discovery, and Integration):** 统一描述、发现和集成协议。 [p.15]
- **协议支持:** 支持多种传输协议，如 HTTP、HTTPS、SMTP、JMS。 [p.15]

### SOAP Web Service 对比 RESTful API (SOAP vs RESTful API)

[p.20–21]

REST 在大部分应用场景中替代 SOAP 的原因：轻量级、高性能、JSON 数据格式简洁易读、适合移动端和微服务架构。 [p.20]

表 1: SOAP Web Service 与 RESTful API 全面对比 [p.21]

| 对比维度 (Aspect)            | SOAP Web Service                  | RESTful API                              |
|--------------------------|-----------------------------------|------------------------------------------|
| 协议 (Protocol)            | 支持 HTTP、SMTP、JMS 等                | 主要基于 HTTP                                |
| 数据格式 (Data Format)       | 主要是 XML                           | JSON、XML (JSON 更常见, JSON is More Common) |
| 消息结构 (Message Structure) | 复杂 (SOAP Envelope)                | 简单 (HTTP + JSON)                         |
| 状态管理 (State Management)  | 可以有状态 (Stateful) 或无状态 (Stateless) | 无状态 (Stateless)                          |

| 对比维度 (Aspect)    | SOAP Web Service                              | RESTful API                                          |
|------------------|-----------------------------------------------|------------------------------------------------------|
| 性能 (Performance) | 由于 XML 解析较慢、性能较低                              | 轻量级 (Lightweight)、性能更好                               |
| 安全性 (Security)   | 内置 WS-Security, 支持 SOAP 认证、加密等                | 需手动实现安全机制, 如 OAuth 2.0、JWT 等                         |
| 适用场景 (Use Case)  | 企业级应用、银行、金融、政府等需要高安全性和事务性 (Transactional) 的场景 | Web 应用、微服务 (Microservices)、移动应用 (Mobile)、物联网 (IoT) 等 |

## REST 简介与六大约束 (REST Introduction & Six Constraints)

### 什么是 REST? (What is REST?)

[p.22–23]

- REST (Representational State Transfer, 表述性状态转移/表现层状态转移) 这个概念由 **Roy Thomas Fielding** 在 2000 年其博士论文中提出。[p.22]
- Roy Thomas Fielding 是 HTTP 规范 (HTTP Specification) 的主要作者之一, Apache HTTP Server 项目的联合创始人, 计算机体系结构 (Computer Network Architecture) 领域的权威。[p.22]
- REST 是符合 Web Service 思想的一种架构设计风格 (**Architectural Style**), 不是标准 (**Not a Standard**)。[p.23]
- 使用 HTTP、URL、XML、JSON、HTML 等广泛流行的标准和协议。[p.23]
- 设计理念: 轻量级 (Lightweight)、跨平台 (Cross-platform)、跨语言 (Cross-language)。[p.23]
- 一个符合 REST 设计原则的架构就可以称为 RESTful 架构。[p.22]

### REST 的六大约束 (Six Constraints of REST)

[p.24]

- ① **客户端–服务器 (Client–Server)**: 关注点分离 (Separation of Concerns), 客户端负责用户界面 (UI), 服务器负责数据存储 (Data Storage), 二者可独立演化 (Evolve Independently)。[p.24]
- ② **无状态 (Stateless)**: 每个请求必须包含所有必要信息, 服务器不保存客户端上下文 (Client Context)。会话状态全部保存在客户端。这是 REST 最核心的约束之一。[p.24]
- ③ **可缓存 (Cacheable)**: 响应应显式标记是否可缓存 (Cacheable or Non-cacheable)。可缓存的响应数据可被客户端复用, 减少客户端与服务器之间的交互, 提升性能 (Performance) 和可伸缩性 (Scalability)。[p.24]
- ④ **统一接口 (Uniform Interface)**: 通过统一的接口契约进行交互, 包含四个子约束:
  - 资源的标识 (Identification of Resources): 通过 URI 唯一标识资源
  - 通过表述操作资源 (Manipulation of Resources Through Representations): 客户端持有资源的表述即可操作资源
  - 自描述消息 (Self-descriptive Messages): 每条消息包含足够的信息描述如何处理
  - **HATEOAS (Hypermedia As The Engine Of Application State)**: 超媒体作为应用状态引擎, 响应中包含相关链接引导客户端进行后续操作

[p.24]

- ⑤ **分层系统 (Layered System)**: 客户端无法辨别连接的是终端服务器还是中间层 (Intermediary), 中间层可提供负载均衡 (Load Balancing)、共享缓存 (Shared Cache) 等功能, 提高系统可伸缩性。[p.24]
- ⑥ **按需代码 (Code on Demand, 可选)**: 服务器可临时向客户端传输可执行代码 (如 JavaScript), 扩展客

户端功能。这是唯一可选的约束。[p.24]

REST 与 RESTful API 的关系 (REST vs RESTful API)

[p.25]

- 如果一个 API 完全遵循上述六大约束或原则，就被称为 RESTful API。[p.25]
- 实际应用中，很多 API 并不完全符合 REST 的所有约束（如 HATEOAS 常被省略），仍然称为 RESTful API。[p.25]
- 实际项目应用中，总结出的一些常用使用方式称为**最佳实践 (Best Practices)**。[p.25]

REST 核心原则总结 (REST Principles Summary)

[p.24–25]

表 2: REST 核心原则 (REST Core Principles) [p.24–25]

| 原则 (Principle)                 | 含义 (Meaning)            | 考试要点 (Key Points)                      |
|--------------------------------|-------------------------|----------------------------------------|
| 面 向 资 源<br>(Resource-Oriented) | 一切皆资源，每个资源有唯一 URI 标识    | URI 使用 名词 复数，如 /users、/orders          |
| 无状态 (Stateless)                | 每个请求自包含全部信息，服务器不保存客户端会话 | 与 SOAP 的重要区别；利于水平扩展                    |
| 统一接口 (Uniform Interface)       | 通过标准 HTTP 方法操作资源，统一交互方式 | GET/POST/PUT/PATCH/DELETE 各司其职         |
| 资源表述 (Representation)          | 资源可以有多种表述形式，客户端通过内容协商获取 | JSON 最常用，也可用 XML；Accept/Content-Type 头 |
| HATEOAS                        | 响应中包含相关链接，引导客户端发现后续操作   | 超媒体作为应用状态引擎；实践中常被省略                    |
| 分层系统 (Layered System)          | 客户端不感知中间层，支持负载均衡和缓存     | 提高系统可伸缩性 (Scalability)                 |
| 可 缓 存<br>(Cacheable)           | 响应应标记是否可缓存，减少重复请求       | 利用 HTTP 缓存机制提升性能                       |

RESTful API 最佳实践 (RESTful API Best Practices)

请求设计 (Request Design)

[p.26]

- **URI 使用名词 (Noun)**，尽量使用复数 (**Plural**)：如 /orders，不使用动词如 /getOrders。[p.26]
- **URI 使用嵌套 (Nested)** 表示关联关系：如 /users/1/orders/3 表示用户 1 的订单 3。[p.26]
- 使用正确的 **HTTP 方法 (HTTP Methods)**：GET、POST、PUT、PATCH、DELETE 等。[p.26]
- 不符合 **CRUD** 的操作，使用以下三种方式之一：[p.26]
  - (1) POST + 动词（如 POST /orders/1/cancel）
  - (2) 查询字符串中带 action 字段（如 POST /orders/1?action=cancel）
  - (3) 设计子资源 (Sub-resource)（如 POST /orders/1/cancellation）

HTTP 请求方法参考表 (HTTP Methods Reference Table)

[p.27, p.29]

表 3: HTTP 方法 (CRUD 映射) 参考表 [p.27, p.29]

| HTTP 方法 | CRUD 操作       | 幂等性<br>(Idempotent) | 说明 (Description)                                         |
|---------|---------------|---------------------|----------------------------------------------------------|
| GET     | SELECT (查询)   | 是 (Yes)             | 从服务器获取资源 (一项或多项); 安全 (Safe) 且幂等; 参数通过 URL 传递             |
| POST    | CREATE (创建)   | 否 (No)              | 在服务器新建一个资源; 非幂等 (Non-idempotent); 多次调用创建多个资源             |
| PUT     | UPDATE (全量更新) | 是 (Yes)             | 更新资源, 客户端提供改变后的完整资源 (Full Resource); 若资源不存在可创建           |
| PATCH   | UPDATE (部分更新) | 否 (No)              | 部分更新资源, 客户端仅提供需改变的部分字段 (Partial Fields); 非幂等             |
| DELETE  | DELETE (删除)   | 是 (Yes)             | 删除资源; 多次删除结果相同 (资源已不存在)                                  |
| HEAD    | — (元数据)       | 是 (Yes)             | 获取资源头部 (元数据, Metadata), 与 GET 类似但不返回响应体 (Response Body)  |
| OPTIONS | — (信息)        | 是 (Yes)             | 获取资源信息, 如某资源有哪些接口 (HTTP 方法) 可以使用, 用于 CORS 预检 (Preflight) |

CRUD 基础对比范例 (CRUD Comparison Example)

[p.29]

- Create (创建): POST /orders — 请求体包含订单 JSON 数据, 返回 201 Created。
- Read (查询列表): GET /orders — 返回订单列表, 200 OK。
- Read (查询单个): GET /orders/{id} — 返回单个订单, 200 OK; 不存在则 404 Not Found。
- Update (全量更新): PUT /orders/{id} — 请求体包含完整订单数据, 返回 200 OK。
- Update (部分更新): PATCH /orders/{id} — 请求体仅包含需修改的字段, 返回 200 OK。
- Delete (删除): DELETE /orders/{id} — 删除订单, 返回 204 No Content。

购物车复杂场景范例 (Complex Shopping Cart Example)

[p.30]

- 复杂操作 (如加入购物车、下单、取消订单等) 不直接映射到标准 CRUD 操作时, 使用 POST + 动词方式或子资源方式处理。 [p.30]
- 示例: POST /cart/items (添加商品到购物车)、POST /orders/{id}/cancel (取消订单)。 [p.30]

响应设计 (Response Design)

[p.31]



- 使用正确的 HTTP 状态码 (HTTP Status Code): 响应内容中应包含详细且正确的 HTTP 状态码。 [p.31]
- 统一响应格式 (Unified Response Format): 响应内容采用统一响应格式, 返回数据使用 JSON。 [p.31]
- 使用 HTTPS 协议: 网络协议建议使用 HTTPS 协议, 而不是 HTTP 协议。 [p.31]
- 身份认证和鉴权 (Authentication & Authorization): 建议使用标准的身份认证和鉴权规范, 如 JWT、OAuth 框架等。 [p.31]
- 包含相关链接 (HATEOAS Links): 在响应内容中, 最好包含相关其他 API 的链接。 [p.31]

HTTP 状态码参考表 (HTTP Status Code Reference Table)

[p.31, p.34-35]

表 4: 常用 HTTP 状态码参考 [p.31, p.34-35]

| 状 态 码<br>(Code)          | 名称 (Name)              | 含义 (Meaning) | 典型场景 (Typical Scenario)                     |
|--------------------------|------------------------|--------------|---------------------------------------------|
| 2xx 成功 (Success)         |                        |              |                                             |
| 200                      | OK                     | 请求成功         | GET 查 询 成 功、<br>PUT/PATCH 更 新 成 功<br>返回最新数据 |
| 201                      | Created                | 资源创建成功       | POST 创建新资源成功, 通常返回新资源的 URI                  |
| 204                      | No Content             | 请求成功, 无返回内容  | DELETE 删除成功, 响应体为空                          |
| 3xx 重定向 (Redirection)    |                        |              |                                             |
| 301                      | Moved Perma-<br>nently | 永久重定向        | 资源 URI 已永久变更, 客户端应更新书签                      |
| 304                      | Not Modified           | 资源未修改        | 缓存相关, 客户端可使用本地缓存副本                          |
| 4xx 客户端错误 (Client Error) |                        |              |                                             |
| 400                      | Bad Request            | 请求参数有误       | 参数校验失败、JSON 格式错误、缺少必填字段                     |
| 401                      | Unauthorized           | 未认证, 需登录     | 未携带有效 Token, 需先登录获取凭证                       |
| 403                      | Forbidden              | 已认证但无权限      | 用户已登录但无权访问该资源 (鉴权失败, Authorization Failed)  |
| 404                      | Not Found              | 资源不存在        | URI 对应的资源未找到 (可能是 ID 错误)                    |
| 405                      | Method Not Allowed     | HTTP 方法不允许   | 对只读资源使用了 POST (如 POST /users/1)             |
| 409                      | Conflict               | 资源冲突         | 并发修改冲突、重复创建同名资源                             |

| 状态码<br>(Code)            | 名称 (Name)              | 含义 (Meaning) | 典型场景 (Typical Scenario)      |
|--------------------------|------------------------|--------------|------------------------------|
| 415                      | Unsupported Media Type | 不支持的媒体类型     | Content-Type 不是服务器支持的格式      |
| 422                      | Unprocessable Entity   | 无法处理的实体      | 参数语义正确但业务逻辑不通过               |
| 429                      | Too Many Requests      | 请求过多         | 触发限流 (Rate Limiting)         |
| 5xx 服务端错误 (Server Error) |                        |              |                              |
| 500                      | Internal Server Error  | 服务器内部错误      | 代码异常、数据库连接失败等未预期错误           |
| 502                      | Bad Gateway            | 网关错误         | 上游服务器返回无效响应 (如 Nginx 代理后端挂了) |
| 503                      | Service Unavailable    | 服务不可用        | 服务器过载或正在维护停机                 |
| 504                      | Gateway Timeout        | 网关超时         | 上游服务器响应超时                    |

统一响应格式范例 (Unified Response Format Example)

[p.34–35]

典型统一响应结构 (JSON 格式): [p.34–35]

```
{
  "code": 200,
  "message": "success",
  "data": { ... }
}
```

- **code:** 状态码 (Status Code), 便于客户端判断请求结果, 通常与 HTTP 状态码保持一致。
- **message:** 提示信息 (Message), 描述性文本, 供前端展示或调试使用。
- **data:** 数据载荷 (Payload), 实际返回的业务数据, 错误时可为 null。
- 统一格式便于客户端统一处理成功和错误两种情况, 无需根据 HTTP 状态码分别解析。[p.34–35]

Spring MVC REST 相关注解 (Spring MVC REST Annotations)

核心注解详解 (Core Annotation Details)

[p.37–39]

- **@RestController** — 类级别注解, 组合了 **@Controller** 和 **@ResponseBody**。标注后该类所有方法返回值自动序列化 (Serialize) 为 JSON 写入 HTTP 响应体, 适用于 RESTful API 开发。与 **@Controller** 的关键区别: **@Controller** 方法返回值默认走视图解析器 (View Resolver) 进行页面跳转; **@RestController** 方法返回值直接写入响应体。[p.37–39]
- **@RequestMapping** — 类/方法级别通用映射注解。可指定 **path** (或 **value**)、**method**、**params**、**headers**、**consumes**、**produces** 等属性。常用于 Controller 类级别设置全局路径前缀。[p.37–39]
- **@GetMapping** / **@PostMapping** / **@PutMapping** / **@PatchMapping** / **@DeleteMapping** — 方法级别快捷注解, 分别对应 HTTP GET/POST/PUT/PATCH/DELETE 方法。是 **@RequestMapping(method = ...)** 的简

- 化写法。[p.37–39]
- **@PathVariable** — 参数级别,绑定 URL 路径中的模板变量 (Template Variable)。如 `@GetMapping("/users/{id}")` 中的 `id` 通过 `@PathVariable("id") Long id` 获取。用于 RESTful URL 的资源标识。[p.37–39]
  - **@RequestParam** — 参数级别, 绑定 URL 查询参数 (Query Parameter) 或表单数据。支持 `required` (是否必填)、`defaultValue` (默认值) 属性。如 `@RequestParam("page") int page`。[p.37–39]
  - **@RequestBody** — 参数级别, 将 HTTP 请求体 (Request Body) 中的 JSON/XML 反序列化 (Deserialize) 为 Java 对象。需配合 `Content-Type: application/json` 使用。常用于 POST/PUT 请求接收复杂数据结构。[p.37–39]
  - **@ResponseBody** — 方法级别, 将返回值序列化 (Serialize) 写入 HTTP 响应体。`@RestController` 已包含此注解, 无需单独标注。[p.37–39]
  - **@ResponseStatus** — 方法级别, 指定返回的 HTTP 状态码。如 `@ResponseStatus(HttpStatus.CREATED)` 配合 POST 方法返回 201。[p.37–39]
  - **@CrossOrigin** — 类/方法级别, 解决跨域请求 (CORS, Cross-Origin Resource Sharing) 问题。可指定 `origins`、`methods`、`allowedHeaders` 等属性。[p.37–39]
  - **@ExceptionHandler** — 方法级别, 处理特定异常并返回自定义响应。定义在 Controller 内仅处理该 Controller 的异常。[p.37–39]
  - **@ControllerAdvice** / **@RestControllerAdvice** — 类级别, 全局异常处理和全局数据绑定。`@RestControllerAdvice = @ControllerAdvice + @ResponseBody`。[p.37–39]

注解对照速查表 (Annotation Quick Reference)

[p.37–39]

表 5: Spring REST 核心注解速查 [p.37–39]

| 注解 (Annotation) | 作用级别 (Scope)        | 核心功能 (Core Function)                          |
|-----------------|---------------------|-----------------------------------------------|
| @RestController | 类 (Class)           | = @Controller + @ResponseBody, RESTful 控制器    |
| @RequestMapping | 类/方法 (Class/Method) | 通用请求映射, 可指定 method、path、headers 等             |
| @GetMapping     | 方法 (Method)         | 映射 GET 请求 = @RequestMapping(method=GET)       |
| @PostMapping    | 方法 (Method)         | 映射 POST 请求 = @RequestMapping(method=POST)     |
| @PutMapping     | 方法 (Method)         | 映射 PUT 请求 = @RequestMapping(method=PUT)       |
| @PatchMapping   | 方法 (Method)         | 映射 PATCH 请求 = @RequestMapping(method=PATCH)   |
| @DeleteMapping  | 方法 (Method)         | 映射 DELETE 请求 = @RequestMapping(method=DELETE) |
| @PathVariable   | 参数 (Parameter)      | 绑定 URL 路径变量, 如 /users/{id}                    |
| @RequestParam   | 参数 (Parameter)      | 绑定 URL 查询参数?key=value                         |



| 注解 (Annotation)       | 作用级别 (Scope)        | 核心功能 (Core Function)       |
|-----------------------|---------------------|----------------------------|
| @RequestBody          | 参数 (Parameter)      | 绑定 HTTP 请求体 JSON → Java 对象 |
| @ResponseBody         | 方法 (Method)         | 返回值序列化为 JSON 写入响应体         |
| @ResponseStatus       | 方法 (Method)         | 指定 HTTP 响应状态码              |
| @CrossOrigin          | 类/方法 (Class/Method) | 允许跨域请求 (CORS)              |
| @ExceptionHandler     | 方法 (Method)         | 处理指定异常，返回自定义错误响应           |
| @RestControllerAdvice | 类 (Class)           | 全局异常处理 + @ResponseBody     |

@RestController 对比 @Controller (RestController vs Controller)

[p.37-39]

表 6: @RestController 与 @Controller 对比 [p.37-39]

| 对比维度 (Aspect)        | @Controller                                                   | @RestController                        |
|----------------------|---------------------------------------------------------------|----------------------------------------|
| 组成 (Composition)     | 仅 @Controller 本身                                              | @Controller + @ResponseBody 组合注解       |
| 返回值处理 (Return Value) | 默认走视图解析器 (View Resolver)，返回视图名或 ModelAndView                  | 返回值直接序列化为 JSON/XML 写入 HTTP 响应体         |
| @ResponseBody 需求     | 需要在方法上单独添加 @ResponseBody 才能返回 JSON                            | 所有方法自动具有 @ResponseBody 效果，无需额外标注       |
| 适用场景 (Use Case)      | 服务端渲染 (SSR) 页面跳转，如 Thymeleaf 模板渲染                             | RESTful API 开发，前后端分离架构，返回 JSON 数据      |
| 典型返回                 | 逻辑视图名 (Logical View Name): 如 "home" → /WEB-INF/views/home.jsp | JSON 字符串: 如 {"code":200, "data":{...}} |

CORS 跨域问题 (Cross-Origin Resource Sharing)

跨域问题的来源 (Origin of Cross-Origin Issue)

[p.40]

- 跨域问题的由来是浏览器的同源策略 (Same-Origin Policy)。[p.40]
- 同源策略 (Same-Origin Policy): 浏览器的安全策略之一，指一个网页的内容只能来自于同一个服务器来源，防止恶意网站窃取数据。[p.40]
- 同源 (Same Origin) 判断标准: 两个 URL 如果协议 (Protocol)、主机名 (Host)、端口 (Port) 均一致则同源，三者中有一个不相同即不同源 (跨域, Cross-Origin)。[p.40]

- 示例：对于基准 URL `http://www.csu.edu.cn/cs/index.html`：[p.40]
  - `http://www.csu.edu.cn/cs1/page.html` — 同源（只有路径不同）
  - `https://www.csu.edu.cn/cs/page.html` — 不同源（协议不同, HTTP vs HTTPS）
  - `http://www.csu.edu.cn:8080/cs/page.html` — 不同源（端口不同, 80 vs 8080）
  - `http://mail.csu.edu.cn/cs1/page.html` — 不同源（主机不同, www vs mail）

## Spring Boot 中解决跨域的三种方式 (Three CORS Solutions in Spring Boot)

[p.41]

- **全局配置 (Global Configuration)**: 通过配置类实现 `WebMvcConfigurer` 接口, 重写 `addCorsMappings(CorsRegistry registry)` 方法, 全局配置 CORS 规则。适用于需要统一管理所有跨域策略的场景。[p.41]
- **局部配置 (Local Configuration)**: 使用 `@CrossOrigin` 注解局部配置某个 Controller 或方法。可以指定 `origins`（允许的源）、`methods`（允许的 HTTP 方法）、`allowedHeaders`（允许的请求头）等属性。适用于特定接口需要跨域的场景。[p.41]
- **过滤器方式 (Filter-based Configuration)**: 通过 `CorsFilter` 针对某些 URL 进行跨域配置。基于 Servlet 过滤器机制, 对所有进入 Web 容器的请求生效, 包括静态资源。[p.41]

## 用户认证 (Authentication) — Token & JWT

### 传统架构中的会话跟踪 (Session Tracking in Traditional Architecture)

[p.43]

- **前后端未分离架构的认证流程**: [p.43]
  - 步骤 1. 前端登录, 发送用户名和密码。
  - 步骤 2. 后端根据用户信息生成一个 `jsessionid`, 并保存 `jsessionid` 和对应的用户 ID 到 Session 中。
  - 步骤 3. 把 `jsessionid` 传给用户, 存入浏览器 Cookie。
  - 步骤 4. 之后浏览器请求带上 Cookie, 后端根据 Cookie 值查询用户, 验证是否过期。
- **前后端分离后的两大问题**: [p.43]
  - 问题 1: 前后端分离后浏览器不能自动使用 Cookie、存在跨域问题（Cookie 跨域限制）, 或根本没有浏览器可用（移动端 App、IoT 设备等）。
  - 问题 2: 服务器进行系统架构后（分布式/集群环境, Distributed/Clustered）, Session 不便于存储和共享。

### Token 机制 (Token-based Authentication)

[p.44]

- Token（令牌）在 Web 应用中特指访问资源的凭据（**Credential**）。[p.44]
- 用户请求登录成功后, 服务器基于某种机制生成一个字符串 — token。[p.44]
- **Token 机制与 Session 机制的核心区别**: 服务器是否存储。Session 在服务器端存储（有状态, Stateful）; Token 通常是无状态的 (Stateless), 服务器不存储, Token 本身自包含用户信息。[p.44]
- 之后请求客户端都带上 token, 服务器根据 token 进行用户的认证。[p.44]
- Token 机制需解决 token 的加密 (Encryption)、解密 (Decryption)、过期 (Expiration) 等问题。[p.44]
- 广义上: SessionId 也是 token 机制的一种实现; Cookie 也可被认为是 token 机制的一种实现。[p.44]

## JWT — JSON Web Token

[p.45–49]

### JWT 概述 (Overview)

[p.45]

- JWT 是目前流行的用户认证 token 解决方案，基于开放标准 **RFC 7519**。[p.45]
- 定义了一种简洁的、自包含的 (Self-contained)、以 JSON 对象形式安全传递信息的方法。[p.45]
- 采用**数字签名 (Digital Signature)**，可以被验证和信任。服务器通过签名验证 token 是否被篡改。[p.45]
- 可选对数字签名进行加密（使用 HMAC 算法密钥、RSA 或 ECDSA 公/私钥机制）。[p.45]
- Spring Security 组件中对 JWT 提供集成支持。[p.45]

## JWT 的结构（三部分, Three Parts）

[p.46–47]

JWT 的 token 字符串由三部分组成，每部分均为一个 JSON 对象，用 Base64 URL 安全编码后以句点 (.) 分隔：**Header.Payload.Signature**。[p.46–47]

- **Header (头部)**: 描述 JWT 自身的元数据 (Metadata)。包含: [p.47]
  - **alg**: 签名算法 (Algorithm)，如 HS256 (HMAC SHA256)、RS256 (RSA SHA256)
  - **typ**: token 的类型，固定为 JWT
- **Payload (载荷/负载)**: 存放实际需要传递的数据，包括标准字段 (Registered Claims) 和自定义字段 (Private Claims)。官方标准字段: [p.47]
  - **iss** (Issuer, 签发人)    **exp** (Expiration Time, 过期时间)
  - **sub** (Subject, 主题)    **aud** (Audience, 使用者/受众)
  - **nbf** (Not Before, 生效时间)    **iat** (Issued At, 签发时间)
  - **jti** (JWT ID, 唯一编号)
- **Signature (签名)**: 对 Header 和 Payload 两部分的签名，防止数据篡改。默认使用 HMAC SHA256 进行签名:
 

```
HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret)。
```

 [p.47]

## JWT 的特点 (Characteristics)

[p.48]

- JWT 默认不加密 (**Not Encrypted by Default**)，但可选加密算法进行加密。可以在生成原始 Token 之后，用加密算法再加密一次。[p.48]
- JWT 不仅可以用于认证 (Authentication)，也可以用于数据的传输 (Data Transfer)。在自定义字段中传输数据可以减少对服务器的访问请求。[p.48]
- **主要缺点 (Major Disadvantage)**: token 只要发出，无法在使用过程中废止或修改 token（没有内置的撤销机制, No Built-in Revocation Mechanism）。一旦签发，在过期前始终有效。[p.48]
- 因 JWT 包含认证信息，为防止泄漏应设置尽可能短的有效时间 (**Short Expiration Time**)。通常通过 **exp** 字段控制，配合 Refresh Token 机制实现续期。[p.48]
- JWT 建议采用 **HTTPS** 协议传输，尽量不使用 HTTP 协议，防止中间人攻击 (Man-in-the-Middle Attack) 窃取 token。[p.48]

## JWT 的使用过程 (Usage Flow)

[p.49]

- **首次请求 (登录阶段, Login Phase)**: [p.49]
  - 步骤 1. 客户端发送账号和密码（登录请求）。
  - 步骤 2. 服务器验证成功，生成 JWT 令牌 (Token) 并返回给客户端。
  - 步骤 3. 客户端保存令牌（如存储在 localStorage、SessionStorage 或内存中）。
- **后续请求 (认证阶段, Authentication Phase)**: [p.49]
  - 步骤 1. 客户端在请求头 (HTTP Header) 中携带 JWT 令牌。通常格式: **Authorization: Bearer <token>**。

步骤 2. 服务器验证令牌（检查签名是否正确、令牌是否过期、nbf/iss/aud 等声明是否合法）。

步骤 3. 验证通过后完成响应，返回请求的资源。

## 用户鉴权与 OAuth 框架 (Authorization & OAuth)

### 鉴权问题来源 (Origin of Authorization Problem)

[p.50]

- **鉴权 (Authorization)**，也叫授权，是指用户访问 API 的权限问题 — 某个用户只能访问某些 API，而不能访问其他 API。[p.50]
  - 鉴权常用于系统的**第三方登录 (Third-party Login)** 的应用场景。[p.50]
  - **认证 (Authentication) 与鉴权 (Authorization) 的区别：**
    - 认证 (Authentication): 验证“你是谁” (Who are you?) — 确认用户身份，如通过 JWT Token。
    - 鉴权/授权 (Authorization): 验证“你能做什么” (What can you do?) — 授予有限访问权限，如 OAuth 2.0。
- [p.50]
- **经典类比 — 快递员送外卖：** [p.50]
    - 小区有门禁系统（如 JWT 用户认证），如何让快递员通过门禁进入小区送外卖，又不必知道业主密码，且唯一权限就是送货？
    - 该案例说明了认证与鉴权的分离需求：快递员需要有限的、临时的访问权限，而非完整的用户凭证。

### 鉴权机制的设计 — 快递员范例 (Authorization Mechanism Design)

[p.51]

- 第 1 步： 门禁系统的密码输入器下面增加一个“获取授权”按钮。快递员需要首先按这个按钮，去申请授权。  
[p.51]
- 第 2 步： 快递员按下按钮以后，业主得知有人正在要求授权，系统会显示该快递员的姓名、工号和所属的快递公司等，业主确认后通知门禁系统放行。[p.51]
- 第 3 步： 门禁系统得到确认后，向快递员颁发一个进入小区的令牌 (Access Token)，只在短期内（比如七天）有效。[p.51]
- 第 4 步： 快递员向门禁系统输入令牌，进入小区。[p.51]

该范例对应 OAuth 授权码模式 (Authorization Code Grant) 的核心思想：第三方应用（快递员）获得有限的、临时的访问权限 (Access Token)，而无需用户的完整凭证（密码）。[p.51]

### OAuth 2.0 鉴权框架 (OAuth 2.0 Authorization Framework)

[p.52]

- OAuth 是一个鉴权 (Authorization) 的**开放标准 (Open Standard)**，在软件开发中广泛使用，目前最新版本是 2.0 (<https://oauth.net/>)。[p.52]
- OAuth 为桌面、手机或 Web 应用提供了一种简单的、标准的方式去访问需要用户授权的 API 服务。[p.52]
- Spring Framework 中提供 **Spring Security OAuth2** 组件，实现了 OAuth 规范，方便在 Spring 项目中集成。[p.52]
- OAuth 2.0 的四种授权模式 (Grant Types):
  - **授权码模式 (Authorization Code)**: 最完整的授权模式，第三方应用通过授权码换取 Access Token。对应快递员范例。
  - **隐式模式 (Implicit)**: 简化版授权码模式，适用于纯前端应用（已不推荐）。
  - **密码模式 (Resource Owner Password Credentials)**: 用户直接提供用户名密码换取 Token（适用于

信任的应用)。

- 客户端模式 (Client Credentials): 应用以自己的名义获取 Token, 不涉及用户。

[p.52]

## 内容协商与 API 版本管理 (Content Negotiation & API Versioning)

### 内容协商 (Content Negotiation)

[p.23, p.37–39]

- 内容协商 (Content Negotiation) 是指客户端和服务端协商确定响应数据的格式 (如 JSON、XML)。基于 HTTP 协议的标准机制。[p.23]
- 客户端通过 Accept 请求头指定期望的响应格式:
  - Accept: application/json — 期望返回 JSON
  - Accept: application/xml — 期望返回 XML
- 服务端通过 Content-Type 响应头声明实际返回的格式。
- 客户端通过 Content-Type 请求头声明请求体的格式 (用于 POST/PUT 请求)。
- Spring MVC 中 @RequestMapping 的 produces 和 consumes 属性用于内容协商:
  - produces = "application/json": 指定方法返回 JSON
  - consumes = "application/json": 指定方法接收 JSON

[p.37–39]

### API 版本管理 (API Versioning)

[p.26]

- API 版本管理的常见策略:
  - URI 路径版本 (URI Path Versioning): /api/v1/users、/api/v2/users, 最直观常用。
  - 查询参数版本 (Query Parameter Versioning): /api/users?version=1
  - 请求头版本 (Header Versioning): 自定义请求头如 API-Version: 1
  - Accept 头版本 (Accept Header Versioning): Accept: application/vnd.company.v1+json
- URI 路径版本最为直观且易于调试, 是 RESTful API 中最常见的版本管理方式。[p.26]

## 本章小结 (Chapter Summary)

[p.53]

### 三大核心主题 (Three Core Themes)

1. 前后端分离的基本概念 (Frontend-Backend Separation):
  - JSP/PHP 时代 (p.5) → MVC 框架时代 (p.6) → 前后端分离时代 (p.7–11)
  - 前后端分离的定义: 解耦 (Decouple), 通过统一 API 通信 (p.8)
  - 优势: 独立部署、并行开发、BFF 层、微服务就绪 (p.9)
2. Web 服务的实现方式 (Web Service Implementation):
  - 基于 SOAP 的 Web Service: XML + WSDL + UDDI、多协议支持、企业级 (p.15–18)
  - 基于 REST 的 RESTful API: HTTP + JSON、轻量级、无状态、Web/移动/物联网 (p.20–25)
  - SOAP vs REST 核心差异: 协议、数据格式、消息结构、状态管理、性能、安全性、适用场景 (p.21)
3. Spring Boot RESTful API 实现 (Spring Boot RESTful API Implementation):
  - 核心注解: @RestController、@RequestMapping、@GetMapping 等 (p.37–39)
  - @PathVariable、@RequestParam、@RequestBody 参数绑定 (p.37–39)



- CORS 跨域三种解决方案：全局配置 / @CrossOrigin / 过滤器 (p.41)
- Token / JWT 用户认证：Header.Payload.Signature 三部分结构、无状态、HTTPS 传输、短有效期 (p.44–49)
- OAuth 2.0 用户鉴权：Access Token、四种授权模式 (p.50–52)

## RESTful API 设计要点速记 (Quick Reference)

- **URI 设计 (URI Design)**: 名词、复数、嵌套表示关联关系；`/users/1/orders/3`
- **HTTP 方法 (HTTP Methods)**: GET (查) / POST (增) / PUT (全量改) / PATCH (部分改) / DELETE (删)
- **响应设计 (Response Design)**: 统一格式 (code + message + data)、JSON、HTTPS、正确状态码、含链接 (HATEOAS)
- **状态码 (Status Codes)**: 2xx 成功 / 3xx 重定向 / 4xx 客户端错误 / 5xx 服务端错误
- **认证 (Authentication)**: JWT (Header.Payload.Signature)、无状态、HTTPS 传输、短有效期
- **鉴权 (Authorization)**: OAuth 2.0、Access Token、四种授权模式
- **安全 (Security)**: CORS 同源策略 (协议 + 主机 + 端口)、@CrossOrigin、全局/局部/过滤器三种方案
- **内容协商 (Content Negotiation)**: Accept / Content-Type 头、produces / consumes 属性
- **API 版本管理 (API Versioning)**: URI 路径版本 / 查询参数 / 请求头 / Accept 头